Name: Cohort/Batch: Date: Score:

GITHUB COPILOT PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Developer Tools

TOTAL: 10 QUESTIONS

Q1 What is GitHub Copilot and what problem in Developer Tools does it solve? Mental Model

Q2 What are the core components or concepts in GitHub Copilot? Architecture

Q3 How does GitHub Copilot compare to its main alternatives? Integration

Q4 How do you get started with GitHub Copilot for a new project? Testing

Q5 What configuration options most affect GitHub Copilot's behavior in production? SSO / IdP

Q6 How does GitHub Copilot handle reliability and high availability? Observability

Q7 What observability does GitHub Copilot provide out of the box? Lifecycle

Q8 What are common performance bottlenecks in GitHub Copilot? Architecture

Q9 What security best practices apply when running GitHub Copilot? Testing

Q10 What mistakes do teams commonly make when adopting GitHub Copilot? Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 GitHub Copilot is a tool or platform

Mitigation

GitHub Copilot is a tool or platform that automates or simplifies a specific concern in the Developer Tools domain, reducing manual effort and operational complexity for engineering teams.

A2 GitHub Copilot has key building blocks that

Primitives

GitHub Copilot has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A3 GitHub Copilot trades off simplicity against feature

Hardening

GitHub Copilot trades off simplicity against feature richness differently than its alternatives. Choose GitHub Copilot when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A4 Install GitHub Copilot via its package manager

Gotchas

Install GitHub Copilot via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A5 Production-critical settings typically include concurrency

Integration

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A6 GitHub Copilot provides HA through leader election,

Observability

GitHub Copilot provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A7 GitHub Copilot exposes metrics (Prometheus endpoint or

Automation

GitHub Copilot exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A8 Performance bottlenecks typically stem from misconfigured

Governance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A9 Run GitHub Copilot with the principle of

Assessment

Run GitHub Copilot with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A10 Common pitfalls include skipping the documentation and

Optimization

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.